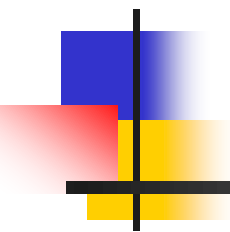


DATA STRUCTURES AND ALGORITHMS



Lecture Notes 4.1

Pradit Pitaksathienkul

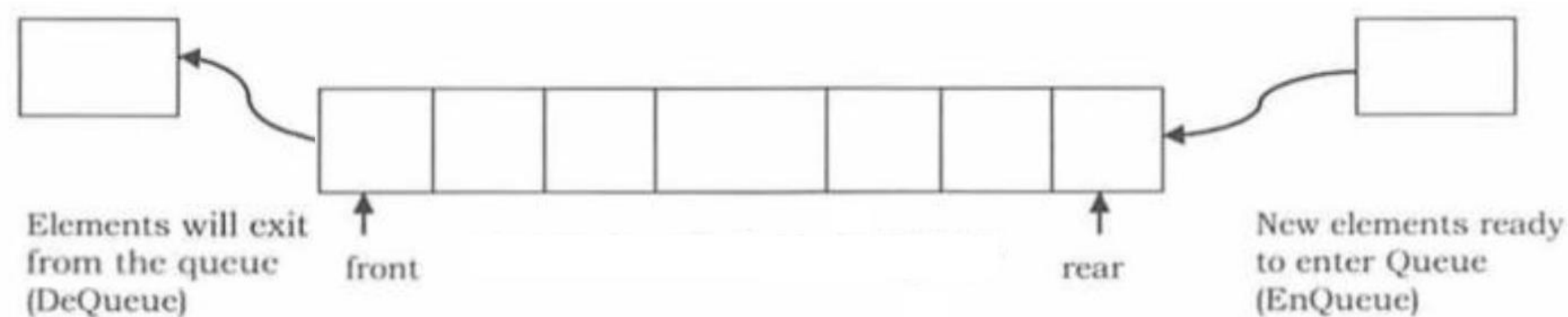


ROAD MAP

- Abstract Data Types (ADT)
 - The List ADT
 - List implementation of lists
 - Linked list implementation of lists
 - The Stack ADT
 - Linked list implementation of stacks
 - List implementation of stacks
 - **The Queue ADT**
 - **List implementation of queues**
 - Linked List implementation of queues

THE QUEUE ADT

- A queue is a list
 - insertions at one end
 - deletions at other end



Operations ?



THE QUEUE ADT

- Operations
 - Enqueue
 - insert an element at the end of the list (rear)
 - Dequeue
 - delete the element at the start of the list (front)
 - IsEmpty
 - check whether the queue has an element or not





Implementation of Queue ADT

- Linked list implementation of queues
 - keep two pointers head and tail
- List implementation of queues (Python)
 - keep positions of front and rear
 - keep track of the size of queue
 - data type list in python
 - has method append and pop already



List Implementation of Queues

- Enqueue → use method `append()`
Increment `queueSize`
Increment `rear`
- Dequeue → use method `pop()`
Decrement `queueSize`
Decrement `rear`

Note: `method pop()` base index 0,
and python shifting the
remaining elements left.

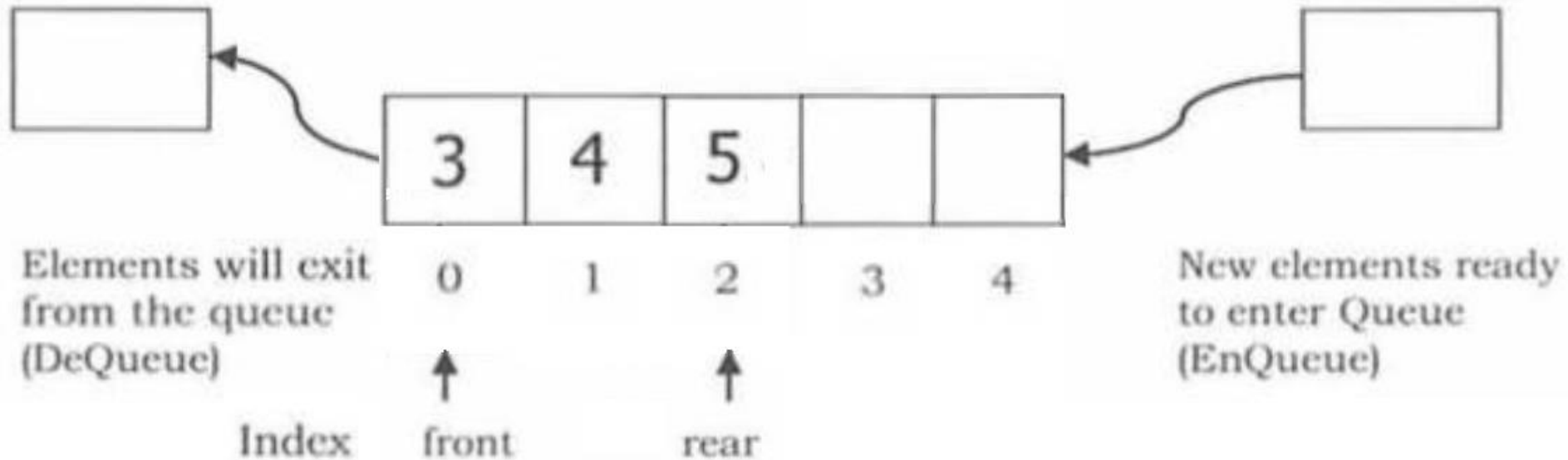
```
class Queue():  
    //Construct the queue.  
    def __init__(self, limit = 5):  
        self.items = []  
        self.limit = limit  
        self.front = 0  
        self.rear = -1  
        self.size = 0  
  
    def isEmpty(self):  
        return self.size < 0  
  
    def enqueue(self, item):  
        self.items.append(item)  
        self.rear += 1  
        self.size += 1
```

```
def deQueue(self) :  
    self.items.pop(0)  
    self.size = self.size-1  
    self.rear = self.size-1  
  
def printqueue(self) :  
    for i in range(self.size) :  
        print(self.items[i], end=' ' )  
    print()
```



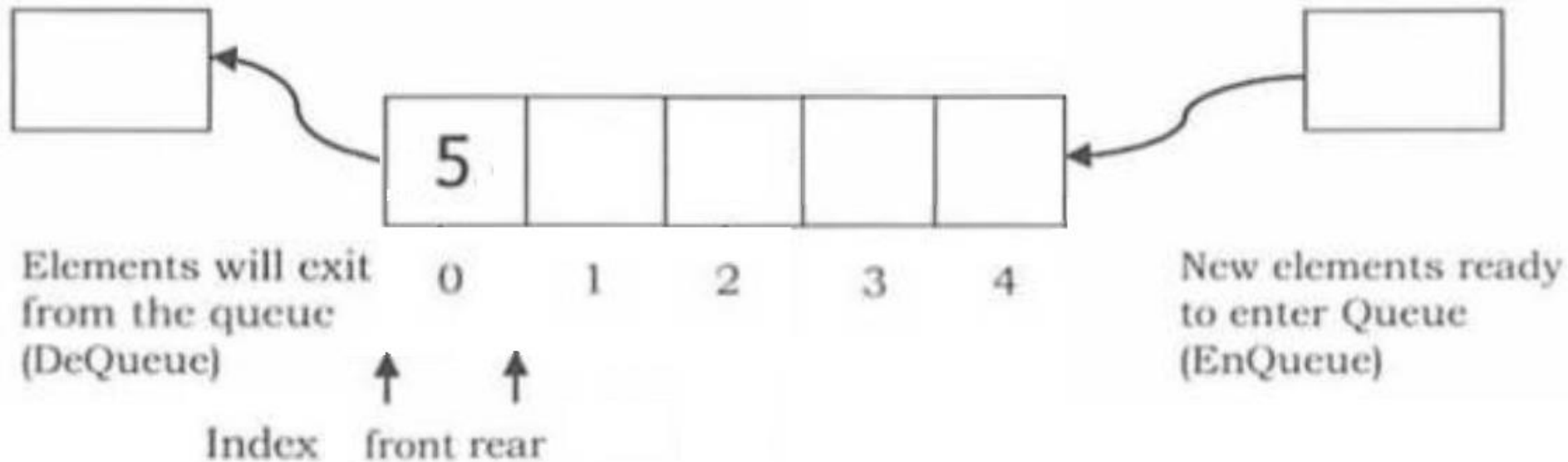
```
myqueue = Queue(5)
myqueue.enqueue(3)
myqueue.enqueue(4)
myqueue.enqueue(5)
```

```
myqueue.printqueue()
```



```
/* Dequeue - Remove item at the front,  
method pop() base index 0,  
and python shifting the remaining elements left.*/
```

```
myqueue.dequeue()  
myqueue.dequeue()  
print("After Dequeue")  
myqueue.printqueue()
```





Special cases of Queues

1. Queue is full, when enqueue
2. Queue is empty, when dequeue

How to solve?